

EN.601.447/647 Computational Genomics: Sequences

Homework 1 (Fall 2026)

Out of 28 points (17 written, 11 programming).

Where a question refers to **Hunter**, it means the “Life and its Molecules” reading, posted here: https://www.cs.jhu.edu/~langmea/teaching/f2021_447/life_and_its_molecules.pdf

Programming questions

1. Reverse complement. (1 pt)

Submit this question as hw1q1.

Because this is the first programming question of the course, it spells out some conventions that will generally hold for every programming question on every homework.

- You may write in **C/C++**, **Java** or **Python**.
- Your program will usually take **two command-line arguments**: the path of an input file and the path of an output file. It will read its input from the first path and write its answer to the second path. Nothing is read from the terminal and nothing is printed to it. Future questions might vary this convention, e.g. by using multiple input files or multiple output files.
- Name your solution according to the name given in the question, plus the suffix for your submission language: `.py`, `.java` or `.cpp`. Submit it to the “HWx Programming” section in Gradescope, where x is the number of the homework.
- When we evaluate your output, we will trim whitespace. Specifically, the autograder ignores blank space at the start and end of each line, and a trailing newline at the end of the file. Our autograder does check that your output has the correct number of lines, so please refrain from printing any extra text to the output file(s).
- Some questions are worth 0 points. Gradescope cannot mark a 0-point question, so it will show a small non-zero value such as 0.01 instead. Ignore that; it does not matter for your recorded grade.

For example, if your solution is in Python and named `hw1q1.py`, we will run it as:

```
python3 hw1q1.py input.txt output.txt
```

Now for the actual question. Write a program that reads a DNA string and writes its *reverse complement* on a single line. Recall that the reverse complement is formed by complementing each base (A with T, C with G), then reversing the result.

For this question only, ignore any input character that is not an uppercase A, C, G or T.

Example. If `input.txt` contains:

ATGACC

then `output.txt` should contain:

GGTCAT

2. **Hamming distance.** (2 pts)

Submit this question as hw1q2.

Write a program that takes two DNA strings of the same length and reports the number of offsets at which the two differ. This is called the “Hamming distance,” which we will return to later. The input consists of two lines, with a DNA string on each. You can assume the input is well formatted: only DNA characters are given, there is no extra whitespace beyond the line separators, and the two strings have the same length.

Your solution must be named `hw1q2`, plus (`.py`, `.java`, or `.cpp`).

INPUT in the `input.txt` file:

GCCAT
CGAAT

OUTPUT in the `output.txt` file:

3

3. **Counting the DNA strings that encode a peptide.** (4 pts)

Submit this question as hw1q3.

Write a program that counts how many distinct DNA strings encode a given peptide. “Peptide” means a sequence of amino acids.

As discussed in the Hunter reading, the genetic code is *degenerate*: 64 codons encode 20 amino acids plus a “stop” codon. Most amino acids have several synonymous codons that code for them. Leucine has six, for example: `UUA`, `UUG`, `CUU`, `CUC`, `CUA` and `CUG`. Note that these are written using the RNA base “U” (uracil) instead of the corresponding DNA base “T” (thymine), which is the more correct way to write them when speaking about RNA.

Your program will read one line from its input file: a peptide written in single-letter amino acid code, always ending in a single `*`, which stands for the stop codon. Your program will write a single integer to the output file, equal to the number of distinct DNA strings that translate to exactly that line.

The `*` is guaranteed to be present once, at the very end of the input. Like most amino acids, the stop codon can be encoded by more than one codon, so it should multiply into your final count too.

Example. If `input.txt` contains:

MA*

then `output.txt` should contain:

12

Methionine (M) is encoded by a single codon, AUG. Alanine (A) is encoded by four: GCU, GCC, GCA and GCG. The trailing `*` is the stop, and it can be any of three: UAA, UAG and UGA. So the number of DNA strings is $1 \times 4 \times 3 = 12$.

The peptide will be at most 20 amino acids long, not counting the trailing `*`. The largest answer that allows is under 1.1×10^{16} , which fits in a signed 64-bit integer, so your program will be able to print the exact answer without resorting to “infinite precision” integers, though you will need to be careful to pick an integer type large enough to hold the answer. In Python any `int` will do; in C/C++ use `long long` and in Java use `long`, because a 32-bit `int` overflows before you reach a peptide of length 12.

You will need to know how many codons encode each amino acid. The standard table is at <https://w.wiki/3xBV>, and the same information in a form that is easy to read from a program is at https://www.cs.jhu.edu/~langmea/resources/RNA_codon_table.csv, whose `Letter` column gives the single-letter amino acid and marks the three stop codons with `*`. You may read that file or hard-code what you take from it.

Note that the table is written in RNA, so its codons use U. That substitution does not change how many codons encode each amino acid, which is the only thing this question needs from the table.

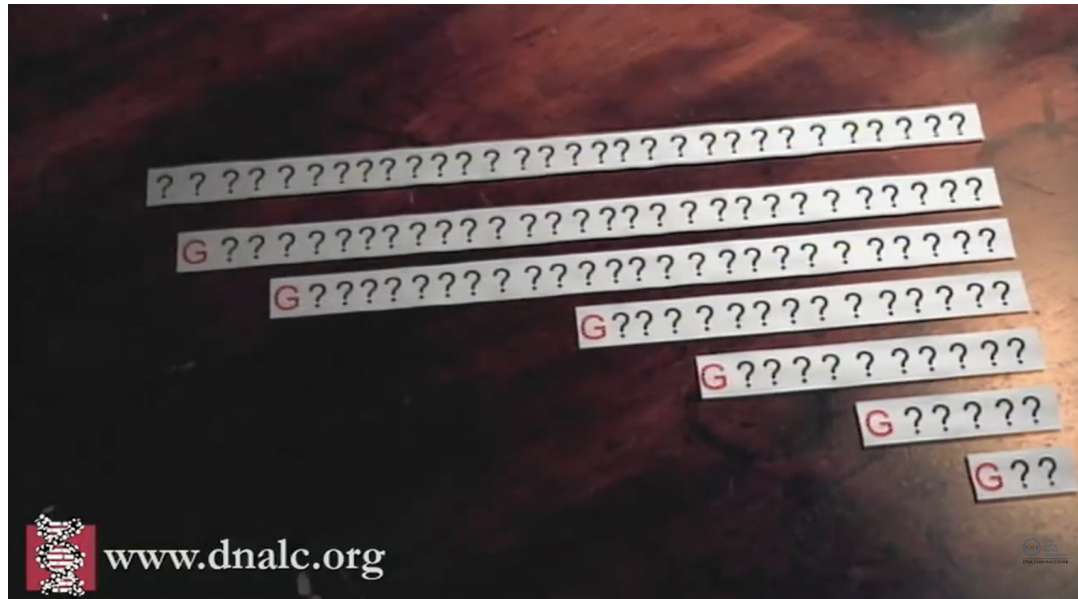
4. Reading a Sanger sequencing ladder. (4 pts)

Submit this question as hw1q4.

Write a program that reconstructs a DNA sequence from a Sanger sequencing “ladder.”

Sanger sequencing is *not* the same as “sequencing-by-synthesis,” which is the dominant technology of today. But Sanger sequencing is important! For instance, it was the method that produced the Human Genome Project’s sequence over the 1990s. Sanger sequencing reads a few hundred bases per reaction, so its “reads” are hundreds of bases long, generally longer than sequencing-by-synthesis reads.

Sanger sequencing works by chain termination. The sequencing reaction is split across four “tubes,” one per base. Laying the fragments out by length turns the collection into a readout of the sequence. The Dolan DNA Learning Center demonstrates this with paper strips, and it is worth watching before you write any code: <https://youtu.be/6ldtdWjDwes>



Fragments from the G tube, as captured from the YouTube video linked above.

As in the video, we consider fragments to be aligned at the right end. Each fragment extends leftward as far as its terminating base. Everything to the right of the terminator is unknown and is written with ?'s.

Offsets in this course are always 0-based, and here that is a statement about the sequence you reconstruct: its first base sits at offset 0 and its last at offset $n - 1$, where n is the length of that sequence.

```

A???
C??
G?
T

```

Reading the terminating bases down the left-hand staircase gives **ACGT**.

Input. Every line is one fragment. You may assume the input always follows this convention: first, the fragments are grouped by tube, assuming the A tube comes first, then C, then G, then T. That is, all the fragments terminating in A will be listed, then all the ones terminating in C, etc. Within each tube grouping, fragments will be listed from longest to shortest. Your program can assume this ordering.

Every line is padded on the left with spaces, so that all of the fragments line up at their **right extreme**. Every line is therefore exactly n characters wide, and the last character of every line sits at the last offset of the sequence; a fragment of length L begins in column $n - L$. The spaces are there to make the ladder readable and carry no information of their own — it is a fragment's *length* that tells you which offset it reveals.

The length n of the sequence is not given, but your program can infer it as the length of the longest provided fragment.

Output. The reconstructed sequence, of length exactly n , on one line. Where the input data fails to identify exactly which base goes at an offset, print **X**. That can happen for two distinct reasons:

- There was no fragment of the length that would reveal the base at that offset.
- There were two or more fragments of the appropriate length, but they disagreed about the terminating base.

Repeated fragments with the same terminating base are not a disagreement. Two identical fragments agree with each other, and the offset is decided as normal; it becomes **X** only if some other fragment of that length terminates in a different base.

Example. This one exercises all three situations at once. If `input.txt` contains:

```
A???  
  A?  
C????  
C????  
G?????  
  T???  
    T
```

then `output.txt` should contain:

```
GCXXAT
```

The longest fragment is `G?????`, of length 6, so $n = 6$. Note that it is neither the first line of the input nor in the A tube: the longest fragment can sit in any tube, so you have to look for it rather than assume where it is. Three offsets are then worth following:

- **Offset 1** is read by the length-5 fragments, and there are two of them. But they both terminate in **C**, so the base at that offset is **C**.
- **Offset 2** is read by the length-4 fragments, and those two disagree; one terminates in **A**, the other in **T**. Because of the disagreement here, we print **X**.
- **Offset 3** would be read by a length-3 fragment, and the ladder has none, so it is **X** as well.

Written questions

1. Repeats in the human genome. (4 pts)

When the Human Genome Project was underway, there was a debate about whether to sequence the genome piece by piece (“BAC-by-BAC”) with a final stitching step, or all at once (“whole-genome shotgun”). Both sides of the debate agreed that what made the assembly problem hard was the presence of *repetitive DNA*. Dr. Philip Green thought this meant that the whole-genome shotgun approach was not practical; he put his objection this way:

“The fundamental difficulty in assembly is dealing with repeats, and bacterial genomes have very few of these.”

In other words, we have been able to sequence and assemble bacterial genomes without a problem because they are not very repetitive. Since human genomes are very repetitive, it will be much harder.

Weber and Myers, arguing the other way, tested feasibility with a simulation that modeled the genome’s repeat content with two assumptions: *20% of genome comprised of SINEs with 300-nucleotide lengths*, and *5% of genome comprised of LINEs with 1500-nucleotide lengths*.

Neither paper explains what a SINE or a LINE is, exactly how much of the human genome they cover (which wasn’t known at the time), or where repeats come from in the first place. Let’s investigate these questions. Do these short readings:

- **Reading 1.** Treangen and Salzberg, “Repetitive DNA and next-generation sequencing: computational challenges and solutions,” *Nature Reviews Genetics* 13:36–46, 2012. **Box 1 only**, which is one page. Prof. Salzberg is on the faculty here, and hosts a free copy of the corrected paper: <https://ccb.jhu.edu/people/salzberg/docs/Treangen-Salzberg-NatRevGen2011-corrected.pdf>
- **Reading 2.** Bourque et al., “Ten things you should know about transposable elements,” *Genome Biology* 19:199, 2018. **First section only**, the one headed “Transposable elements come in many different forms and shapes.” Open access: <https://doi.org/10.1186/s13059-018-1577-z>

- (1 pt) Box 1 of Treangen and Salzberg opens with a single figure for how much of the human genome is repetitive. What is it?
- (1 pt) The table in Box 1, panel (a), lists seven classes of repeat. Which class occurs in the largest number of copies, and how many? Which class covers the largest fraction of the genome, and what fraction?
- (1 pt) Bourque et al. divide transposable elements into two major classes according to their mechanism of transposition. Name both classes and give each mechanism in a phrase. Then say which class LINEs and SINEs belong to.
- (1 pt) Return to the Weber and Myers assumptions quoted above (the 5% and 20%). Compare those assumptions with the corresponding rows of Box 1’s table. Which of the two did they get more wrong, and by roughly how much?

2. Templates, terminators and copying DNA. (6 pts)

These questions cover how DNA is copied and how sequencing by synthesis eavesdrops on that process to produce sequencing reads.

- (a) (2 pts) Explain why it makes sense to call a single-stranded segment of DNA a *template*. Your answer should say what property of the molecule makes the word apt, and it should make clear why the same word would not be apt for a protein.
- (b) (2 pts) In sequencing by synthesis, each nucleotide added to the growing strand carries a “terminator” group that must be removed before the next base can be added. This deliberately slows the reaction down. What is the terminator for, and what would go wrong without it?
- (c) (2 pts) Sequencing by synthesis reads a *cluster* of identical template molecules, not a single one. Give one reason the cluster is necessary, and one problem that using a cluster introduces which sequencing a single molecule would not have.

3. Palindromes. (2 pts)

A DNA sequence is called a *palindrome* when it is equal to its own reverse complement. For example, GAATTC is a palindrome: complementing each base gives CTTAAG, and reversing that gives back GAATTC. Note that this is not the everyday sense of the word; GATTAG is the same backwards as forwards, but is not equal to its reverse complement, so we would not call it a palindrome.

- (a) (1 pt) How many length-4 DNA palindromes are there?
- (b) (1 pt) Is a length-3 DNA palindrome possible? If so, write one as your answer. If not, explain why it is not possible.

4. The wobble base. (5 pts)

Hunter observes that alanine is encoded by GCT, GCC, GCA and GCG, and that for most amino acids the synonymous codons differ only in the third nucleotide. That third position is often called the *wobble base*.

You will want the genetic code table for this question: <https://w.wiki/3xBV>, or the same information as a file at https://www.cs.jhu.edu/~langmea/resources/RNA_codon_table.csv. The table is written in RNA; you may answer in DNA or RNA, but say which.

- (a) (3 pts) There are $64 \times 3 = 192$ ways to mutate some codon’s wobble base, counting each codon paired with each of the three bases it does not already have in that position. What fraction of those 192 result in the codon coding for a *different* amino acid? Give your answer as a fraction.
For this count, treat “stop” as though it were one more amino acid: a change that creates a stop, or destroys one, *does* count as changing the amino acid, while a change from one stop codon to another does not.
- (b) (2 pts) As Hunter notes, not every organism and organelle uses the same genetic code. Do some reading online and report one specific case where a codon that is a *stop* codon in the standard code instead codes for an amino acid. Give the codon, the amino acid, and the organism or system in which it happens, and cite where you found it.

5. **Special review problem: tracing naive exact matching.** (0 pts)

This problem is not graded: it is worth 0 points on this homework. It is here because a problem similar to this one will appear on Midterm 1, where it will be worth about 4 of the exam's points, out of around 45 total points. A solution to this problem will be posted along with the rest of the solutions for this homework. But please try to answer this before then so that it functions as real practice for you.

Let $T = \text{GTCTGCACGACGATCGTC}$ and $P = \text{ACGA}$. Following the midterm's convention, write $n = |P|$ and $m = |T|$. Offsets are 0-based: $T[0]$ is the first character of T , and the alignment starting at offset s begins by comparing $P[0]$ to $T[s]$.

- (a) How many *alignments* does naive exact matching try? Answer in terms of n and m first, then give the integer number. Then give the total number of *character comparisons* naive matching makes, using the typical nested loops, with the inner loop comparing characters left to right, as we discussed in lecture. Count every iteration of the inner loop, whether the characters match or mismatch.
- (b) What are the smallest and largest numbers of character comparisons naive matching could possibly make, for a T and P of these lengths? Give a concrete T and P of the same lengths achieving each.
- (c) At the alignment starting at offset 6 every character of P matched (ACGA matched ACGA). Using *only* the text characters you have already compared, which of alignments 7, 8 and 9 can you immediately rule out? For each alignment you rule out, say which character comparison you can guarantee will mismatch.